PARCIAL 1

SISTEMAS OPERATIVOS

Universidad Sergio Arboleda

Ciencias de la computación e Inteligencia Artificial

Docente:

Ivan Dario Mendez Aguilera

Integrantes:

Brayan Camilo Moreno Cordero

Bogota

9 de Febrero del 2026

1. ESTRUCTURA DE LA ENTREGA

```
Parcial2_BrayanMoreno/
├── Punto1_Gramatica_CRUD/
│   └── gramatica_crud.txt
├── Punto2_ANTLR/
│   ├── CRUD.g4
│   ├── main.py
│   ├── CRUDLexer.py
│   ├── CRUDParser.py
│   └── CRUDListener.py
├── Punto3_Demostracion_LL1/
│   ├── demostracion_ll1.py
│   └── salida_demostracion.txt
├── Punto4_Comparacion_CYK_LL1/
│   ├── cyk_parser.py
│   ├── ll1_parser.py
│   ├── comparar.py
│   └── comparacion_parsers.png
├── Punto5_Calculadora_Booleana/
│   ├── boolean_calc.y
│   └── captura_ejecucion.txt
└── README.md
```

2. PUNTO 1 – GRAMÁTICA CRUD (DISEÑO)

Archivo: Punto1_Gramatica_CRUD/gramatica_crud.txt

Nota: Este punto es únicamente de diseño teórico. No requiere ejecución.

Gramática utilizada (formato simple):

```
<programa>      ::= <instruccion> { <instruccion> }

<instruccion>   ::= <operacion> ( ";" | "\n" )

<operacion>     ::= <create_op>
                  | <read_op>
                  | <update_op>
                  | <delete_op>

<create_op>     ::= "create" "(" <id> "," <diccionario> ")"
<read_op>       ::= "read" "(" <id> "," <diccionario> ")"
<update_op>     ::= "update" "(" <id> "," <diccionario> "," <diccionario> ")"
<delete_op>     ::= "delete" "(" <id> "," <diccionario> ")"

<diccionario>   ::= "{" [ <pares> ] "}"

<pares>         ::= <par> { "," <par> }

<par>           ::= <string> ":" <valor>

<valor>         ::= <string>
                  | <numero>
                  | <booleano>
                  | <diccionario>

<id>            ::= [a-zA-Z_][a-zA-Z0-9_]*
<string>        ::= "\"" { <caracter> } "\""
<numero>        ::= [ "-" ] <digito> { <digito> } [ "." <digito> { <digito> } ]
<booleano>      ::= "true" | "false"
```

Ejemplo válido:

```
create(users, {"name": "Juan", "age": 30});
read(users, {"age": 30});
update(users, {"name": "Juan"}, {"$set": {"age": 31}});
delete(users, {"active": false});
```

3. PUNTO 2 – IMPLEMENTACIÓN EN ANTLR

Gramática utilizada (formato simple):

```
Reglas del parser:
programa → instruccion programa | ε
instruccion → operacion ( ';' | NL )
operacion → create_op | read_op | update_op | delete_op
create_op → CREATE '(' ID ',' diccionario ')'
read_op   → READ   '(' ID ',' diccionario ')'
update_op → UPDATE '(' ID ',' diccionario ',' diccionario ')'
delete_op → DELETE '(' ID ',' diccionario ')'
diccionario → '{' pares? '}'
pares → par ( ',' par )*
par → STRING ':' valor
valor → STRING | NUMERO | BOOLEANO | diccionario

Reglas del lexer (tokens):
CREATE   → 'create'
READ     → 'read'
UPDATE   → 'update'
DELETE   → 'delete'
BOOLEANO → 'true' | 'false'
ID       → letra (letra | digito | '_')*
STRING   → " ( ~["\\] | '\\' . )* "
NUMERO   → -? digito+ ( '.' digito+ )?
NL       → '\n'
WS       → [ \t\r]+ → ignorar
```

Requisitos y ejecución:

```
pip install antlr4-python3-runtime
cd Punto2_ANTLR

java -jar antlr-4.13.2-complete.jar -Dlanguage=Python3 CRUD.g4
python main.py
```

Salida esperada:

```
ANALIZADOR SINTÁCTICO CRUD

Entrada:

    create(usuarios, {"nombre": "Brayan", "id": 101});
    read(usuarios, {"id": 101});
    update(usuarios, {"id": 101}, {"nombre": "Brayan Moreno"});
    delete(usuarios, {"id": 101});


RESULTADO:
  Estado: EXITOSO
  Total instrucciones: 4
  Operaciones encontradas: CREATE, READ, UPDATE, DELETE

¡Prueba exitosa! El lenguaje reconoció todas las operaciones CRUD.
```

4. PUNTO 3 – DEMOSTRACIÓN LL(1)

Gramática a analizar:

S → A a A b
S → B b B a
A → ε
B → ε

Cálculo de conjuntos:

No terminal	PRIMERO	SIGUIENTE
A	{ε}	{a, b}
B	{ε}	{a, b}
S	{a, b}	{$}

Producción	PREDICCIÓN
S → A a A b	{a}
S → B b B a	{b}
A → ε	{a, b}
B → ε	{a, b}

Verificación LL(1):

- No terminal S: {a} ∩ {b} = ∅ → disjuntos.
- No terminal A: una sola producción.
- No terminal B: una sola producción.

Conclusión:

La gramática ES LL(1).

```
brayan-moreno@brayan-moreno-HP:~/Documentos/UNIVERSIDAD/PARCIAL/punto3$ python3 explicacion.py
GRAMÁTICA:
  S -> A a A b
  S -> B b B a
  A -> ε
  B -> ε

RESUMEN DE CONJUNTOS:
PRIMEROS: {'S': {'b', 'a'}, 'A': {'ε'}, 'B': {'ε'}}
SIGUIENTES: {'S': {'$'}, 'A': {'b', 'a'}, 'B': {'b', 'a'}}
PREDICCIÓN: {'S -> A a A b': {'a'}, 'S -> B b B a': {'b'}, 'A -> ε': {'b', 'a'}, 'B -> ε': {'b', 'a'}}

JUSTIFICACIÓN DE CONDICIÓN LL(1)

Para el no terminal S:
   Producción 1: S → A a A b → PRED = {'a'}
   Producción 2: S → B b B a → PRED = {'b'}
   ✓ Los conjuntos PREDICCIÓN son disjuntos.

El no terminal A tiene una sola producción → no hay conflicto.

El no terminal B tiene una sola producción → no hay conflicto.

CONCLUSIÓN:
La gramática ES LL(1) porque, para cada no terminal, los conjuntos PREDICCIÓN
de sus producciones son disjuntos. Esto significa que con un solo símbolo
de anticipación se puede decidir de manera única qué producción aplicar.
brayan-moreno@brayan-moreno-HP:~/Documentos/UNIVERSIDAD/PARCIAL/punto3$
```

5. PUNTO 4 – COMPARACIÓN CYK vs LL(1)

Gramática original (referencia):

E → E + T | E - T | T
T → T * F | T / F | F
F → (E) | num

Gramática en CNF (para CYK):

E → E A1 | T
A1 → + T | - T
T → T A2 | F
A2 → * F | / F
F → P E C | num
P → (
C →)

Gramática LL(1) (para parser predictivo):

E → T E'
E' → + T E' | - T E' | ε
T → F T'
T' → * F T' | / F T' | ε
F → (E) | num

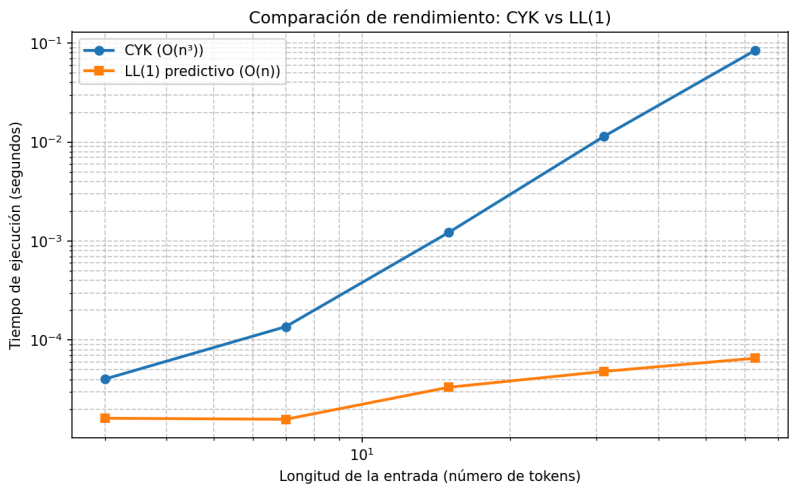
Resultados de rendimiento:

Tokens	CYK (s)	LL1 (s)	Aceleración (CYK/LL1)
3	0.000026	0.000010	2.6x
7	0.000096	0.000010	9.6x
15	0.000677	0.000014	48.4x
31	0.006295	0.000033	190.8x
63	0.048481	0.000049	985.1x

Gráfica: comparacion_parsers.png (escala log-log) muestra pendiente ~3 para CYK (cúbica) y ~1 para LL(1) (lineal).

```
brayan-moreno@brayan-moreno-HP:~/Documentos/UNIVERSIDAD/PARCIAL/punto4$ python3
comparar.py
Comparación CYK vs LL(1)

n=  3 | CYK: 0.000040s | LL1: 0.000016s | válido=False
n=  7 | CYK: 0.000137s | LL1: 0.000016s | válido=False
n= 15 | CYK: 0.001222s | LL1: 0.000033s | válido=False
n= 31 | CYK: 0.011402s | LL1: 0.000048s | válido=False
n= 63 | CYK: 0.084695s | LL1: 0.000066s | válido=False
```



Conclusión:

LL(1) es O(n), adecuado para tiempo real. CYK es O(n³), impracticable para entradas largas.

6. PUNTO 5 – CALCULADORA BOOLEANA EN YACC

Gramática utilizada (formato simple):

```
input → line input | ε
line → expr '\n' { printf("%d\n", $1); }
expr → TRUE | FALSE
     | expr '|' expr
     | expr '&' expr
```

| '!' expr
| '(' expr ')'

TRUE → 'true'
FALSE → 'false'

Compilación y ejecución:

yacc -d boolean_calc.y
gcc -o boolean_calc y.tab.c
./boolean_calc

Ejemplo de uso:

true & false → 0
true | false → 1
!true → 0
(true & false) | true → 1
false | (true & true) → 1

Desempeño del analizador:

- Método: LALR(1) generado por YACC.
- Complejidad: O(n) lineal.
- Responde en microsegundos.
- Directivas %left, %right resuelven precedencia y asociatividad eficientemente.

Conclusión:

Muy eficiente y adecuado para calculadora de escritorio interactiva.

```
brayan-moreno@brayan-moreno-HP:~/Documentos/UNIVERSIDAD/PARCIAL/punto5$ yacc -d boolean_calc.y
gcc -o boolean_calc y.tab.c
brayan-moreno@brayan-moreno-HP:~/Documentos/UNIVERSIDAD/PARCIAL/punto5$ ./boolean_calc
Calculadora booleana (true/false, and/or/not, Ctrl+D para salir)
true
1
not true
0
true & false
true | false
!true
(true & false) | true
false | (true & true)
true
false0
1
0
1
1
1
```